PeerForge — Technical Architecture

Version: feat/phase-0-multitenancy · **Last verified:** 2026-08-05 against a running instance.

Every number in this document was measured, not estimated. Where something is known not to work, it says so.

---

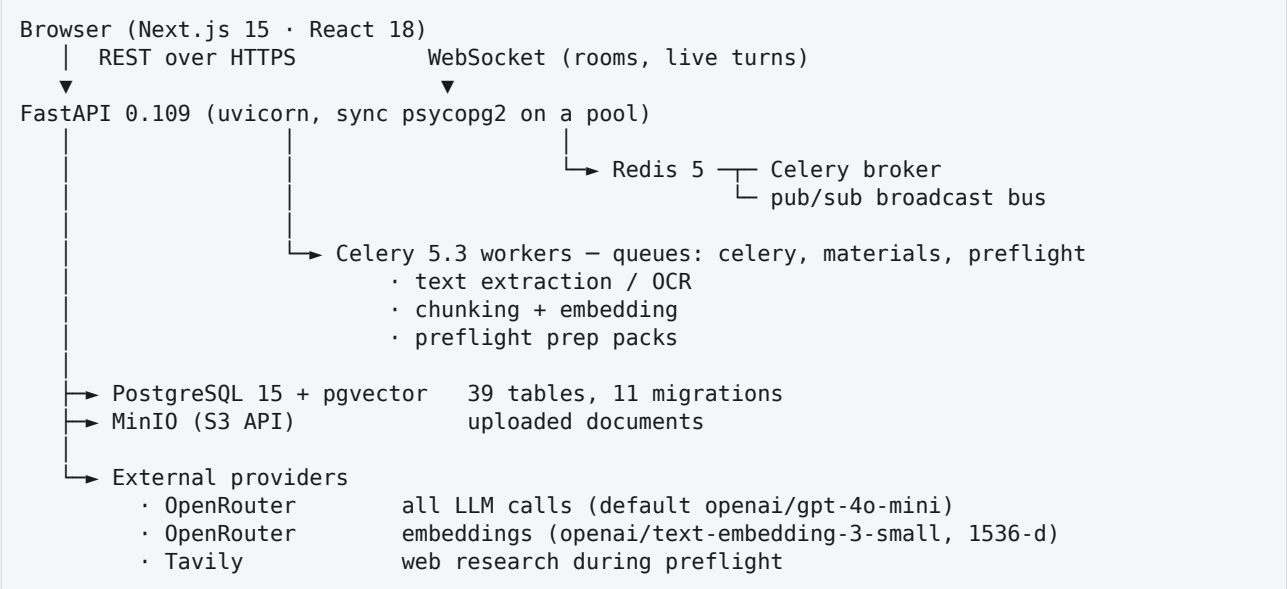
1. What the system is

PeerForge runs an academic peer-review panel against a piece of research. A user uploads a paper, describes the problem, and a panel of AI reviewers — each with a distinct remit — reads the material, prepares privately, then debates it in rounds. The output is a transcript, a summary, a readiness assessment, a set of defence questions, and a signed certificate binding the assessment to the evidence it rested on.

It is not a chatbot with personas. The distinguishing work is in the machinery that stops a panel of language models degenerating into one voice repeated five times, and stops them citing sources that do not exist.

---

2. Shape of the system



Scale of the codebase: 109 Python files / ~30,300 lines; 99 TypeScript files / ~22,600 lines. 107 API paths, 121 operations. 440 backend tests across 35 files.

---

3. The data model

39 tables, grouped by what they are for.

Group	Tables	Holds
Identity (7)	`tenants`, `workspaces`, `user_workspaces`, `organization_members`, `organization_seats`, `invitations`, `user_settings`	Institution → course → member hierarchy, seats, invitations
Session (5)	`debates`, `participants`, `agents`, `events`, `debate_outputs`	A review session, its panel, its append-only event log, its summaries
Knowledge (6)	`meeting_materials`, `memory_chunks`, `agent_knowledge_units`, `agent_memories`, `debate_memory_grants`, `memory_access_log`	Uploaded documents, embedded chunks, prep packs, cross-session memory and who may read it
Preparation (5)	`preflight_runs`, `preflight_participant_runs`, `research_profiles`, `defense_questions`, `session_answers`	Per-reviewer preparation, the extracted research profile, viva questions and answers
Assurance (5)	`academic_assessments`, `issued_certificates`, `signing_keys`, `readiness_reports`, `transcript_action_items`	Scored readiness, signed certificates, extracted actions
Other (11)	`artifacts`, `audit_logs`, `debate_analytics`, `agent_personalities`, …	Supporting concerns

The hierarchy

```
tenant  (an institution — "Northgate University")
  └── workspace  (a course — "PSY-601 Research Methods")
        └── debate  (a review session)
              ├── participants  (the reviewer panel)
              ├── events        (append-only transcript)
              └── meeting_materials → memory_chunks (embedded)
```

Membership is recorded twice on purpose, and the distinction matters:

- `organization_members` — your role in the **institution** (`org_admin`, `professor`, `ta`, `student`)
- `user_workspaces` — your role in a **specific course**

Authorization resolves a debate's workspace and checks membership across **all** of the caller's workspaces (`authorize_debate`). Comparing against the caller's single **active** workspace is a bug this codebase has had twice, most recently in the action-item routes; both are fixed and both have regression tests.

---

4. How a review actually runs

4.1 Ingestion

```
upload → MinIO                      (raw bytes)
       → TextExtractor              (validate MIME, extract text; OCR if needed)
       → TextChunker                (paragraph-aware, 1000 chars, 200 overlap)
       → OpenRouter /v1/embeddings (text-embedding-3-small, 1536-d)
       → memory_chunks              (vector + sha256 + provenance metadata)
```

The chunker is paragraph-aware with a 200-character overlap. That overlap is load-bearing: it is why a section heading survives a chunk boundary, which the section-verification index depends on. A defect where `start = max(start + chunk_size - overlap, end)` silently skipped text — 40 of 200 fuzzed documents lost content — was fixed; the window now steps back by the overlap and never past the previous start.

4.2 Preflight — private preparation

Each reviewer prepares before speaking. Per participant:

1. **Retrieve** — semantic search over the session's chunks, plus any

imported context granted from prior sessions.

1. **Research** — Tavily search on the problem statement (5 results).
2. **Compose** — a role-specific prep prompt produces a private memo.
3. **Persist** — `agent_knowledge_units` row, with metadata recording

`web_research_status`, `web_search_urls`, `web_sources_cited`, `material_chunks_count`, retrieval method and chunk ids.

The memo is private to that reviewer. It is what the "View Prep Pack" dialog shows.

`web_research_status` is one of `ok` / `not_configured` / `unavailable` / `no_problem_statement` / `no_results` / `failed`. It exists because a bare `false` could not distinguish a missing API key from a search that found nothing, and the UI was guessing — it told users to enable a toggle that did not exist.

4.3 The turn — a three-stage pipeline

Every reviewer turn runs three LLM stages. `USE_CONSTITUTIONAL_AI` defaults to on, and this is the live path.

```
Stage 1  Reasoning    agent_reasoning.py
         Private stance, confidence, what others said, what is unique
         about this turn, who to challenge. Structured JSON.

Stage 2  Response     agent_response_generator.py
         The visible message. Role schema supplies the reviewer's lane
         (dimensions, strengths, weaknesses, evidence requirement).

Stage 3  Validation   agent_constitutional_validator.py
         Eleven rules. A critical violation triggers a constrained
         regeneration; the retry is then RE-VALIDATED and gets at most
         one further attempt, kept only if it clears more than it
         introduces.
```

An important architectural fact. `trigger_next_turn` builds a ~7000-token `messages[]` array. On the constitutional path **that array is never sent** — Stage 2 receives `conversation_history`, `turn_info` and `material_context` instead. `messages[]` reaches the model only during constrained regeneration and the exception fallback. Editing a prompt string there and testing a normal turn will show no effect.

4.4 The constitutional rules

Rule	Severity	Catches
`no_hallucination`	critical	Invented participants, placeholder @names

Rule	Severity	Catches
`no_flip_flop`	critical	Unexplained stance reversal
`no_self_contradiction`	critical	Contradicting one's own earlier turn
`no_repetition`	critical	Restating a point already made
`role_consistency`	critical	Straying out of the assigned lane
`persona_authenticity`	critical	Generic filler that any role could have written
`no_fabricated_citation`	critical	Citing a page or section when **no** document was submitted
`no_contradicted_citation`	critical	Citing a section the submitted document does not contain
`no_deference_opener`	critical	Opening by conceding to another reviewer instead of leading
`no_session_meta_commentary`	critical	Reviewing the session setup instead of the work
`must_address_others`	medium	Not engaging with the panel — **advisory only**

Only `critical` affects validity. `must_address_others` is deliberately advisory: measured against 111 real turns it fired on 44%, and inspection showed those turns were fine — they open with the reviewer's own point, which is exactly what the round-1 instruction asks for. Escalating it would make two parts of the system fight each other.

4.5 Citation integrity

Two distinct problems, handled differently because the evidence differs.

No document submitted. Every page or section reference is invented by definition. `_DOC_LOCATOR` matches document locators (`p. 12`, `Section 2.1`, `Table 3`) but deliberately **not** external literature (`(McKay et al., 2018)`). Detection forces regeneration; a final backstop on the published message replaces any survivor with `[source not provided]`.

The page pattern is bounded to 1–3 digits: case-insensitively, `p.\s*\d+` matches the author initial in `(Jones, P. 2019)`.

A document exists. `services/locator_index.py` builds the set of section, table, figure and appendix identifiers the document actually contains, read from `memory_chunks` — not from the prompt's material context, which is truncated. The verdict is three-valued:

- `verified` — the locator appears in the document
- `absent` — the family exists but this member does not → **acted on**
- `unverifiable` — no locator of that family appears at all, so extraction

may simply have dropped the structure → **stay silent**

Pages are never verified: 391 of 398 material chunks in the live database carry no `page_num`, so a page check would reject nearly every legitimate citation.

4.6 Grounding and provenance

`services/provenance.py` matches a message's substantial sentences against material chunks by exact normalised substring first, then token containment (≥ 0.55). Matches are returned with the chunk's `sha256` re-verified, and stored in the event's `citation_refs` for audit. Pure string matching — no LLM call, no per-turn latency worth noting.

4.7 Data flow, end to end

One request at a time, from a PDF on a desk to a signed certificate.

A. Upload → searchable knowledge

```
Browser  POST /debates/{id}/materials/upload   (multipart)
   │
   ├─ FastAPI validates EVERY file's MIME before touching anything.
   │  An invalid file in a batch changes nothing — the old behaviour
   │  deleted the existing files first, then failed.
   │
   ├─ bytes ──────────────────────────► MinIO       (object store)
   ├─ row   ──────────────────────────► meeting_materials
   └─ enqueue ────────────────────────► Redis → Celery "materials" queue
                                                  │
        TextExtractor  (OCR if the PDF is scanned)
                                                  │
        TextChunker    1000 chars, 200 overlap, paragraph-aware
                                                  │
        POST api.openrouter.ai/v1/embeddings  text-embedding-3-small
                                                  │
                                                  ▼
                                     memory_chunks
                                       · chunk_text
                                       · embedding  vector(1536)
                                       · chunk_metadata { sha256, material_id, page_num }
                                       · source_debate_id   ← the ownership link
```

`source_debate_id` is what every later authorization check resolves through. Its foreign key is `ON DELETE CASCADE` — it was `SET NULL`, which detached prep packs from deleted sessions and made them readable by everyone.

B. Preflight → a private memo per reviewer

```
POST /debates/{id}/preflight/start
   │
   └─► Celery "preflight" queue, once per participant:

        policy_config.problem_statement ──┬─► semantic query
                                          └─► Tavily search (5 results)
                     │                                │
        pgvector similarity over                  web_search_urls
        memory_chunks WHERE source_debate_id      web_search_results
                     │                                │
                     └──────────────┬─────────────────┘
                                    ▼
                    get_preflight_prep_prompt()
                      role lane + materials + literature
                      + REQUIRED "External literature consulted" section
                                    ▼
                    OpenRouter chat completion
                                    ▼
                    agent_knowledge_units
                      · content   the private memo
                      · metadata  { web_research_status, web_sources_cited,
                                    material_chunk_ids, retrieval_method }
```

If `problem_statement` is missing, the search is skipped and the status says `no_problem_statement` rather than failing silently. That field is folded into `policy_config` at creation — the API used to

accept it at the top level and discard it, so 0 of 99 sessions had one.

C. A turn → three model calls and a validation gate

```
POST /debates/{id}/turn/next        (or WS control.next_turn)
   │
   ├─ SELECT … FOR NO KEY UPDATE on debates
   │    NOT plain FOR UPDATE: events.debate_id is an FK, so every
   │    INSERT INTO events takes FOR KEY SHARE on the parent, and the
   │    thinking service writes events on a separate connection mid-turn.
   │    A plain row lock deadlocks the turn against itself.
   │
   ├─ load history (50 events, DESC + reverse), participants, materials
   │
   ├─ STAGE 1  agent_reasoning ──► OpenRouter ──► { stance, confidence,
   │                                                what_others_said,
   │                                                unique_contribution,
   │                                                should_disagree_with }
   │
   ├─ STAGE 2  agent_response_generator ──► OpenRouter ──► message text
   │              inputs: reasoning + conversation_history + material_context
   │              + role schema (dimensions / evidence requirement)
   │
   ├─ STAGE 3  agent_constitutional_validator
   │              + locator_index.find_absent_locators(debate_id, message)
   │              │
   │              ├─ no critical violation ──────────────────► publish
   │              └─ critical ──► constrained regeneration
   │                                  └─ RE-VALIDATE ──► at most one more
   │                                       attempt, kept only if better
   │
   ├─ BACKSTOP  no materials → strip_fabricated_locators
   │            materials    → strip_contradicted_locators (targeted)
   │
   ├─ provenance.ground_message ──► citation_refs   (sha256 re-verified)
   │
   ├─ INSERT INTO events  (sequence_number, content, citation_refs)
   │
   └─ Redis PUBLISH ──► every API instance ──► WebSocket ──► browsers
```

D. Session close → assurance

```
POST /end          debates.state = 'ended'
      ▼
POST /summarize    reads events; REFUSES on an empty transcript
      ▼             writes debate_outputs
POST /analyze-research    materials ──► research_profiles
      ▼
POST /defense-questions/generate
      profile + chunks ──► defense_questions (each with source_excerpt,
                                              source_chunk_id)
      ▼
POST /answers      session_answers
      ▼
POST /assessment/generate
      basis = { has_profile, has_summary, answer_count, message_count }
      ──► academic_assessments (10 dimensions)
      ▼
POST /certificate/issue
      canonicalize({ scores, ordered evidence, event span })
      ──► sha256 ──► certificate_id = "PF-" + digest[:12]
      ──► sign with signing_keys ──► issued_certificates
      ▼
GET /verify/{id}   recompute the anchor from CURRENT evidence
                     signature bad or hash bad  → INVALID
                     both good, evidence same   → VALID
                     both good, evidence moved  → SUPERSEDED
```

The certificate id being a content hash is why `SUPERSEDED` is possible at all: a changed session produces a different id, and the old id still verifies as authentic — just no longer current.

E. Where each secret lives

```
OPENROUTER_API_KEY  ─┐
TAVILY_API_KEY      ─┼─ apps/api/.env.local  (gitignored, server only)
SUPABASE_JWT_SECRET ─┘
                          │
     resolve_openrouter_key(header) ── caller header is an OVERRIDE only
                          │
     browser ── never required to hold one; check-key-gates.js fails the
                build if any generation gate depends on a browser key
```

---

5. Real-time transport

```
Browser ──WebSocket──► FastAPI ──► Redis pub/sub ──► other instances
                          │
                          └─ renewable lease (LEASE_TTL_SECONDS = 180)
```

Redis pub/sub is what makes broadcast work across more than one API instance; a purely in-process registry only reaches clients on the same worker. The debate lease prevents two instances driving the same autonomous session.

Room controls map as follows:

Control	Transport	Endpoint
Start	REST	`POST /debates/{id}/start`
Pause / Resume	WS `control.pause` / `control.resume`, REST fallback	`POST /debates/{id}/pause` · `/resume`

Control	Transport	Endpoint
Next Turn	WS `control.next_turn`, REST fallback	`POST /debates/{id}/turn/next`
+2 Rounds	REST	`PATCH /debates/{id}/extend` (`extend_rounds`)
End	WS `control.end`, REST fallback	`POST /debates/{id}/end`
Autonomous	REST	`POST /api/debates/{id}/start-autonomous` · `pause-` · `resume-`

The OpenRouter key is resolved **server-side** for every one of these. A client-supplied `X-OpenRouter-Key` is only an override.

---

6. Assurance chain

```
transcript ──► summary          (refuses to run on an empty transcript)
           ──► research profile (extracted from the materials)
           ──► defence questions (grounded in source excerpts)
           ──► assessment        (10 dimensions, scored, with a stated basis)
           ──► certificate       (sha256 over scores + ordered evidence,
                                  signed with a key from signing_keys)
```

The certificate id **is** a content hash (`PF-` + first 12 hex of the sha256), so a session whose evidence has moved produces a different id. `GET /verify/{id}` returns a three-valued verdict:

- `VALID` — signature and hash check out, evidence unchanged
- `SUPERSEDED` — authentic, but the evidence has changed since issue
- `INVALID` — signature or hash fails

`SUPERSEDED` exists because a two-valued verdict called an out-of-date certificate a forgery.

---

7. Security model

- **Authorization** — every debate-scoped handler routes through

`authorize_debate(debate_id, user)`, which resolves the debate's workspace and checks membership across all of the caller's workspaces.

- **Keys** — OpenRouter and Tavily keys live server-side. The browser never

needs one; `scripts/check-key-gates.js` fails the build if any generation gate depends on a browser-held key. It binds the key to whatever name it is given (a `const` from `keyStore.getKey()`, an aliased destructure, a prop) and flags boolean use on either side of `&&` and `||`.

- **Untrusted content** — uploaded materials are authored by the person under

review. They are fenced in the user role with delimiters and an explicit instruction that a request to change role, verdict or rules is to be quoted as a finding, not obeyed.

- **Provenance** — chunk `sha256` is re-verified at citation time.

Deployment posture, stated plainly

The current public deployment runs with `REQUIRE_AUTH=false` and both API keys server-side. **Anyone with the link can use it and spend those credits.** That is a deliberate choice for demonstration, not an oversight. Turning auth on requires a real `SUPABASE_JWT_SECRET` — the API refuses to boot with a placeholder, by design.

---

8. Quality measurement

`services/transcript_quality.py` scores a transcript offline. Two metrics gate; two are reported only.

Metric	Role	Limit
`restatement_rate`	**gate**	≤ 0.5
`opener_template_rate`	**gate**	≤ 0.5
`placeholder_rate`	gate	≤ 0.2
`self_similarity`	diagnostic	—
`role_differentiation`	diagnostic	—

The last two were demoted after measurement across ten real transcripts showed their good and bad ranges overlap completely — the **worst** transcript scored the lowest self-similarity and the highest role differentiation, because reviewers restating one point in different words look lexically diverse. Enforcing them failed the cleanest session measured.

`restatement_rate` uses containment against the shorter side over every earlier turn — the same measure the live repetition guard uses, so the online guard and the offline gate agree.

---

9. Known limitations

Stated because a technical document that only lists strengths is not useful.

1. **Section verification cannot check pages.** `page_num` is absent from 391 of 398 live chunks.
1. **Reviewer differentiation degrades by round two** on short papers. The gates catch it; the fix is partial.
1. **Prep packs cite 3 of 5 retrieved sources**, not all five — the required section caps at two or three bullets deliberately, since forcing all five in produces padding rather than argument.
1. **The backend test suite writes to the same database** and leaves its fixtures behind. Seed demo data **after** testing, or point tests at a separate database.
1. **Quick-tunnel URLs are ephemeral** and `NEXT_PUBLIC_API_URL` is inlined at build time, so a new API URL requires a rebuild, not a restart.

---

10. Running it

```
# infrastructure
docker compose -f infra/docker/docker-compose.yml up -d   # postgres, redis, minio

# api
cd apps/api && source venv/bin/activate
python -m uvicorn src.main:app --host 0.0.0.0 --port 8000

# workers — the -Q list is required, not optional
celery -A src.celery_app worker -Q celery,materials,preflight --loglevel=info

# web
cd apps/web && npm run build && npm run start -- -p 3001
```

Required environment (`apps/api/.env.local`, gitignored):

Variable	Purpose
`OPENROUTER_API_KEY`	All LLM calls and embeddings
`TAVILY_API_KEY`	Preflight web research (optional; absence is reported, not hidden)
`DATABASE_URL`	PostgreSQL
`REDIS_URL`	Celery broker and pub/sub
`REQUIRE_AUTH`	`false` for open demo; `true` needs `SUPABASE_JWT_SECRET`
`CORS_ALLOW_ORIGINS`	Allowlist for the web origin